

# **IMPLEMENTATION AND ANALYSIS OF MACHINE LEARNING TECHNIQUES IN eBPF FOR NETWORK SECURITY**

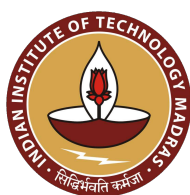
A PROJECT REPORT

submitted by

**Saran Kumar**  
(CS20B069)

to

the Indian Institute of Technology, Madras  
in partial fulfillment of the requirements for the award of the Degree  
of  
Bachelor of Technology  
in  
Computer Science & Engineering



**Department of Computer Science & Engineering**

Indian Institute of Technology, Madras

Chennai

600036

June 2024

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

**COMPUTER SCIENCE & ENGINEERING**

**Indian Institute of Technology, Madras**

**Chennai**

**600036**



**CERTIFICATE**

This is to certify that the report entitled **Implementation and Analysis of Machine Learning techniques in eBPF for network security** submitted by **Saran Kumar** to the Indian Institute of Technology, Madras in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science & Engineering is a bonafide record of the project work carried out by him under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide

Name: Prof. Krishna Moorthy Sivalingam

Signature: .....

Head of Department

Name: Prof. Krishna Nandivada

Signature: .....

## ACKNOWLEDGMENT

I wish to record my indebtedness and thankfulness to all who helped me prepare this Project Report titled **Implementation and Analysis of Machine Learning techniques in eBPF for network security** and present it satisfactorily.

I am especially thankful for my guide and supervisor Prof. Krishna Moorthy Sivalingam in the Department of Computer Science & Engineering for giving me valuable suggestions and critical inputs during the course of this project and for the preparation of this report.

My friends and lab-mates from DAWNLAB have always been helpful and I am grateful to them for their help related to the Project. Special thanks to Madhan Raj Kanagarathinam, PhD scholar at DAWNLAB, for his help and guidance and also to Dr. Palanivel Kodeswaran and Dr. Sayandeep Sen at IBM India for their insights and feedback.

Saran Kumar

(Reg. No. CS20B069)

Bachelor of Technology

Department of Computer Science & Engineering

Indian Institute of Technology, Madras

## ABSTRACT

In this report, the implementation of machine-learning techniques at kernel-level using eBPF (extended Berkeley Packet Filter) for network security in order to obtain an improvement in latency of querying the ML model is documented.

eBPF is a revolutionary technology with origins in the Linux kernel that can run sandboxed programs in a privileged context such as the operating system kernel. It is used to safely and efficiently extend the capabilities of the kernel without requiring to change kernel source code or load kernel modules. As eBPF programs run within the Linux kernel itself, allowing them to process packets efficiently without the overhead of transitioning to user space. This kernel-level integration enables eBPF to provide high-performance networking capabilities. We aim to leverage these capabilities to build a low-latency Intrusion Detection System in the Linux kernel. Network security is heavily reliant on fast-processing and low-latency systems in order to detect malicious packets before they attack the system. Thus, we choose this use case and implement an IDS using ML techniques. Our system utilizes eBPF's programmability to design a custom decision tree model that can efficiently classify network packets as benign or malicious based on a variety of attributes and metadata.

Apart from testing the feasibility of implementing complex programs in eBPF, we aim to observe an improved performance in terms of latency of the eBPF program when compared to the same IDS being implemented in the userspace. To evaluate the effectiveness of our approach, we conduct extensive experiments using real-world datasets. The results demonstrate that the eBPF in-kernel implementation has much lower latency compared to the userspace while achieving an accuracy score of 94%.

# CONTENTS

|                                               |           |
|-----------------------------------------------|-----------|
| <b>ACKNOWLEDGMENT</b>                         | <b>i</b>  |
| <b>ABSTRACT</b>                               | <b>ii</b> |
| <b>LIST OF TABLES</b>                         | <b>iv</b> |
| <b>LIST OF FIGURES</b>                        | <b>v</b>  |
| <b>Chapter 1. INTRODUCTION</b>                | <b>1</b>  |
| 1.1 Overview and Problem Statement . . . . .  | 2         |
| 1.1.1 Complexity of eBPF solutions . . . . .  | 2         |
| 1.1.2 Problem Statement . . . . .             | 3         |
| 1.2 Motivation . . . . .                      | 3         |
| 1.3 Contributions . . . . .                   | 4         |
| 1.4 Organization of this Report . . . . .     | 5         |
| <b>Chapter 2. BACKGROUND AND RELATED WORK</b> | <b>6</b>  |
| 2.1 What is eBPF? . . . . .                   | 6         |
| 2.2 How are eBPF programs compiled? . . . . . | 11        |
| 2.2.1 Clang . . . . .                         | 11        |
| 2.2.2 BCC . . . . .                           | 12        |
| 2.2.3 LIBBPF . . . . .                        | 14        |
| 2.2.4 BPFTRACE . . . . .                      | 15        |
| 2.3 Maps . . . . .                            | 17        |
| 2.4 Helper Functions . . . . .                | 17        |
| 2.5 TC Interface . . . . .                    | 18        |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Chapter 3. PROPOSED CLASSIFIER IN EBPF</b>      | <b>20</b> |
| 3.1 Data Collection . . . . .                      | 20        |
| 3.2 Offline Training of DT Model . . . . .         | 24        |
| 3.3 Loading DT Model . . . . .                     | 26        |
| 3.4 Challenges Faced . . . . .                     | 27        |
| 3.4.1 Choice of eBPF Toolchain . . . . .           | 27        |
| 3.4.2 Design of DT . . . . .                       | 29        |
| 3.4.3 Debugging . . . . .                          | 30        |
| <b>Chapter 4. PERFORMANCE STUDY</b>                | <b>31</b> |
| 4.1 Dataset Used . . . . .                         | 31        |
| 4.2 Design and Performance of eBPF Model . . . . . | 31        |
| 4.2.1 Classification Report . . . . .              | 32        |
| 4.2.2 Latency . . . . .                            | 33        |
| <b>Chapter 5. CONCLUSIONS AND FUTURE WORK</b>      | <b>36</b> |
| 5.1 Conclusions . . . . .                          | 36        |
| 5.2 Future Work . . . . .                          | 37        |
| <b>REFERENCES</b>                                  | <b>38</b> |

## LIST OF TABLES

|     |                                   |    |
|-----|-----------------------------------|----|
| 4.1 | Split of Friday dataset . . . . . | 31 |
| 4.2 | Classification Report . . . . .   | 33 |
| 4.3 | Average Time Taken . . . . .      | 35 |

## LIST OF FIGURES

|     |                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------|----|
| 2.1 | The kernel provides system calls through which userspace applications can make requests . . . . . | 7  |
| 2.2 | Loading and attaching to an event of an eBPF program (bytecode) . . . . .                         | 10 |
| 2.3 | Stages of compilation of an eBPF program . . . . .                                                | 11 |
| 2.4 | Clang compiler with target bpf specified outputs eBPF bytecode . . . . .                          | 12 |
| 2.5 | BCC compiler overview . . . . .                                                                   | 13 |
| 2.6 | Workflow of libbpf compilation . . . . .                                                          | 15 |
| 2.7 | bpftool compilation overview . . . . .                                                            | 16 |
| 2.8 | eBPF maps accessed by eBPF programs and userspace application . . . . .                           | 17 |
| 3.1 | Contents of struct <code>__sk_buff</code> . . . . .                                               | 21 |
| 3.2 | Declaration of eBPF map <code>PUC_info</code> . . . . .                                           | 22 |
| 3.3 | Entries of struct <code>PUC_entry</code> . . . . .                                                | 23 |
| 3.4 | Workflow of Data Collection from eBPF and userspace program . . . . .                             | 24 |
| 3.5 | Trained Decision Tree diagram . . . . .                                                           | 25 |
| 3.6 | Error message for an eBPF program . . . . .                                                       | 30 |
| 4.1 | Confusion Matrix . . . . .                                                                        | 32 |
| 4.2 | Span of total time for in-kernel implementation of model . . . . .                                | 34 |
| 4.3 | Span of total time for userspace implementation of model . . . . .                                | 35 |



# Chapter 1

## INTRODUCTION

eBPF originated from the Berkeley Packet Filter (BPF), which was designed in 1992 as a domain-specific in-kernel virtual machine for efficient packet capture. The most pivotal moment in BPF's evolution came with Linux Kernel 3.18, which introduced eBPF (Extended Berkeley Packet Filter). eBPF incorporated major enhancements like custom data structures (maps), helper functions, and tail calls. With this, came the expansion of eBPF to new use cases, allowing tracing and manipulation of kernel function calls, syscalls, and other system events with minimal overhead. In Linux 4.8, XDP (eXpress Data Path) was introduced, enabling eBPF programs to run at the earliest networking driver level, providing a framework for extremely high-performance packet processing. eBPF has seen widespread adoption in computing, with major companies like Facebook, Google, and Netflix using it due to its performance benefits. Every Android phone uses eBPF to monitor traffic, and every packet going in and out of a Facebook datacenter is touched by eBPF.

The use-case we have chosen is network security, as this benefits greatly from low latency and high throughput. Detecting and mitigating malicious network traffic is a critical aspect of modern cybersecurity defences. Traditional methods of packet inspection often struggle to keep pace with the evolving threat landscape, leading to delays in identifying and responding to malicious activity. In this project, we propose a novel approach to packet inspection using a Decision Tree model implemented in eBPF for real-time detection of malicious packets within the Linux kernel. Our system leverages the flexibility and performance benefits of eBPF to embed a Decision Tree classifier directly into the network stack, enabling efficient and low-latency analysis of incoming packets. By defining decision rules based on

packet features, our system can accurately classify packets as either benign or malicious with minimal computational overhead. The Decision Tree model is trained on labelled packet data to learn patterns indicative of malicious behaviour, allowing the system to generalise and detect previously unseen threats effectively. Through continuous monitoring and analysis of network traffic, our system can proactively identify and block malicious packets in real time, enhancing the overall security of the network. By combining the power of machine learning with the efficiency of kernel-level packet inspection, we offer a scalable and effective solution for enhancing network security and mitigating cyber threats with low latency.

## **1.1 OVERVIEW AND PROBLEM STATEMENT**

Along with the overview of the problem, this section will discuss the complexity of the problem we choose to solve, the context of the proposed solution with respect to state-of-the-art solutions, and define the problem statement.

The main aim of this project is to implement a machine learning (ML) model within eBPF framework to classify network packets as either malicious or benign. Leveraging the advancements of eBPF, the project seeks to evaluate the feasibility and effectiveness of embedding ML models directly into eBPF programs. Traditionally, ML models for network security are deployed in userspace, which can introduce latency due to context switching between kernel space and userspace. By integrating the ML model within eBPF, we aim to achieve real-time packet classification with reduced latency, thereby improving the overall efficiency of network security mechanisms.

### **1.1.1 Complexity of eBPF solutions**

Implementing solutions in eBPF, particularly integrating Machine Learning (ML) models, presents a unique set of challenges due to the inherent constraints and design principles of eBPF. The complexity arises from the need to adapt ML algorithms to fit within the limitations of eBPF, such as the absence of external li-

braries, restrictions on floating-point operations, and the passive event-based model of eBPF programs. We are required to overcome these limitations by translating complex ML algorithms into eBPF-compatible code, often involving the use of fixed-point arithmetic and custom data structures to represent decision trees or neural networks. Additionally, the process involves training ML models externally and then transferring them into eBPF-compatible code, which demands careful consideration of memory access boundaries and efficient data processing techniques to ensure optimal performance within the kernel environment. Despite these challenges, the integration of ML models in eBPF offers significant benefits, such as improved packet processing accuracy, scalability, and reduced latency, making it a promising avenue for enhancing network intelligence and security capabilities.

### **1.1.2 Problem Statement**

Design and implement an efficient and scalable Intrusion Detection System that leverages the capabilities of extended Berkeley Packet Filter (eBPF) to accurately classify network packets as benign or malicious by integrating ML models for improved classification accuracy and adaptability, while ensuring high-performance processing and low latency as well as evaluating the feasibility of implementing complex solutions in eBPF framework.

## **1.2 MOTIVATION**

The primary motivation for this project is to explore and leverage the evolving capabilities of the extended Berkeley Packet Filter (eBPF) framework for real-time network security applications. eBPF has emerged as a powerful tool within the Linux kernel, allowing the execution of custom code in response to various events such as network packet arrivals. Its ability to operate at the kernel level, combined with recent advancements in eBPF technology, presents a unique opportunity to enhance the performance of machine learning (ML) models used in network security.

**Testing eBPF's Expanding Functionality:** As eBPF continues to evolve, its po-

tential applications are expanding beyond traditional use cases such as performance monitoring and tracing. This project aims to test and validate eBPF's ability to support complex ML algorithms for real-time security applications. Successfully implementing an ML model within eBPF will not only demonstrate the framework's versatility but also pave the way for future innovations in the field.

**Addressing Latency Issues:** One of the significant challenges in deploying ML models for network security is the latency introduced by userspace operations. Traditional implementations require frequent context switches between the kernel and userspace, resulting in delays that can hinder the timely detection and mitigation of malicious activities. By embedding ML models directly into eBPF programs, this project aims to minimise these latencies, thereby enabling faster and more efficient packet classification.

**Enhancing Network Security:** In today's digital landscape, cyber threats are becoming increasingly sophisticated and pervasive. The need for robust and responsive network security mechanisms has never been greater. Integrating ML models into eBPF can significantly enhance the capability to detect and respond to threats in real-time. This integration is particularly crucial for high-speed networks where even minor delays can have substantial security implications.

### 1.3 CONTRIBUTIONS

Our approach towards the problem statement is to implement a Decision Tree model in eBPF. A main challenge while writing complex programs in eBPF is that eBPF is not Turing-complete. Hence, we choose decision trees (DTs) as they are a simple and effective ML model for Intrusion Detection Systems (IDSs) [3]

We obtain training data using another eBPF program that stores the features obtained from network packets in an eBPF map, which is later read by a userspace program and written onto a CSV file. As we have already written an eBPF program that collects the training data, extending the developed IDS to update the weights of the decision tree dynamically for improved accuracy and more likely to stop zero-day attacks would be easier to build.

Most of the current work and uses of eBPF are for observability, packet processing, performance monitoring, and load balancing. In each of these use-cases, machine learning is likely to be used for better performance. But these ML/AI services are present in the user-space and hence, incorporate a latency to move from kernel-space to user-space. We aim to leverage the functionalities of eBPF to the next level by implementing the ML service required for a said use-case in eBPF itself, in order to avoid this latency. At a bigger level, we aim to step closer to an end-to-end software implemented wholly in eBPF itself and thus, avoiding overhead and improving latency.

The novelty in our approach lies in the fact that most other works involve eBPF for observability and monitoring and not as a separate service itself. Our approach to implementing complex programs in eBPF paves the road for end-to-end services written entirely in eBPF. Another aspect of novelty in this approach is that this allows the incorporation of ML into the Linux kernel. With eBPF, we are able to incorporate ML into the Linux kernel dynamically. The contribution of this project is significant in terms of the result as well, as we have obtained a significant improvement in latency (nearly 5x), strongly suggesting that the current implementations can be extended to solve a much wider set of use-cases.

## **1.4 ORGANIZATION OF THIS REPORT**

The rest of the report is organised as follows:

- Chapter 2: In this chapter, we discuss the vast background and the related work done in eBPF and the context of the same for our use-case.
- Chapter 3: We discuss our algorithm, approach, and eBPF programs in detail and their functionality in this chapter. All the details of the implementation are provided.
- Chapter 4: In this chapter, we infer the results provided by our implementation and analyse its performance.
- Chapter 5: We provide conclusions for the report in this chapter and mention the various directions in which the project can be extended/explored.
- Chapter 6: References used in the report.

## Chapter 2

# BACKGROUND AND RELATED WORK

In this chapter, we give an overview of some of the background material that is required to understand the project. The eBPF framework contains various components and the restrictions posed by the verifier allow us to discuss various workarounds and how to tackle these constraints. We also discuss the related work done in this field and the insights they provide along with how this project is different from other works.

### 2.1 WHAT IS eBPF?

eBPF is a revolutionary kernel technology that allows developers to write custom code that can be loaded into the kernel dynamically, changing the way the kernel behaves.

**Origins of eBPF:** The roots of eBPF lie in the development of BSD Packet Filter (BPF), first described in 1993 in a paper[1] written by Lawrence Berkeley National Laboratory's Steven McCanne and Van Jacobson. It discusses about a program that can filter network packets, written in a general-purpose instruction set of 32-bit instructions that closely resembles assembly language. BPF was first introduced to Linux in 1997, in kernel version 2.1.75 [2], where it was used in the tcpdump utility as an efficient way to capture the packets to be traced out. This was the first step in evolving BPF from the narrow scope of packet filtering to the general-purpose platform it is today.

**Evolution of BPF to eBPF:** BPF evolved to what we know today as extended BPF (eBPF) starting in kernel version 3.18 in 2014, involving significant changes such as the introduction of eBPF maps, the bpf() system call, helper functions along with the eBPF verifier to ensure the eBPF programs are safe to run in the kernel envi-

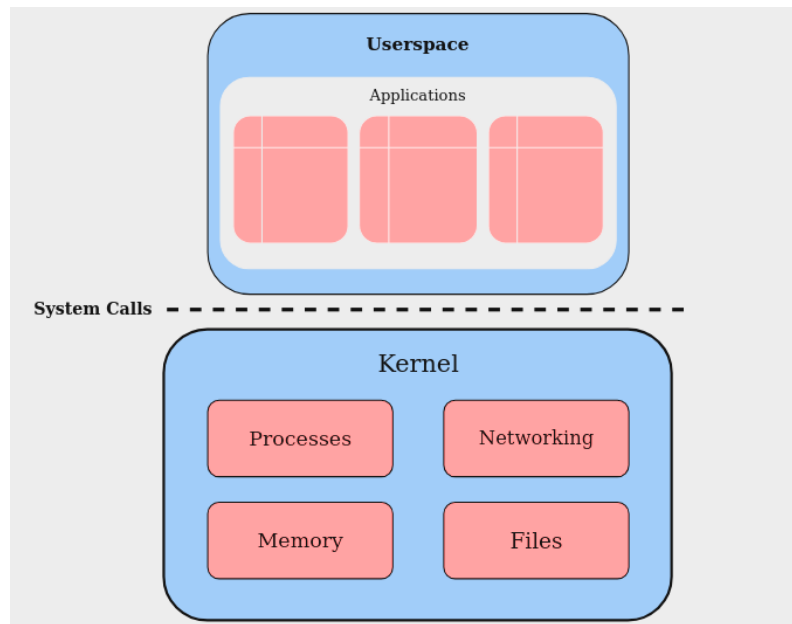


Figure 2.1: The kernel provides system calls through which userspace applications can make requests

ronment. There are still constraints in place to ensure the safety of the kernel, but it is far more robust than the classic BPF implementation. eBPF enables a much wider variety of use cases ranging from high-performance networking to system observability, storage troubleshooting, and more. It retains all of the advantages of the original idea behind BPF; it makes the kernel more programmable without the inherent performance penalties, instability, and insecurity of other approaches. However, it does so with a greatly expanded set of tools, resources, and instructions, which makes it applicable to a much wider set of use cases. To understand the functionalities of eBPF and its place in modifying the kernel, we shall discuss a little about the kernel.

**Modifying Linux Kernel:** The Linux kernel is mostly a closed box and most of the interactions with the kernel occur through system calls, as shown in Figure 2.1. There is very little room for imposing a behaviour change of the kernel unless a feature is requested to the developers, which in turn requires the approval of the community and a time-taking process. By the time your feature is incorporated into the kernel, your requirements may have changed. An alternative to this approach is by using Linux kernel modules, which can be loaded or unloaded on demand. If

required to modify the kernel or extend kernel behaviour, kernel modules are one way to do it. The challenge in writing kernel modules is that these modules are full-on kernel programming and hence, any error or misfit of the module in the kernel would bring the entire kernel down. There can be security concerns as well, with respect to user-written kernel modules. Hence, a big challenge in this aspect is that users cannot easily be sure if their module is safe, not only in terms of crashing the kernel but also in terms of security. This is also another reason why Linux distributions take a long time for new releases.

To overcome these issues, the eBPF technology provides a robust solution: *the eBPF verifier*. The sole purpose of the verifier is to make sure that the eBPF program to be loaded is safe in the context of the kernel, and that it, as described in [4], won't crash the machine or lock it up in a hard loop, and it won't allow data to be compromised.

**The Verification Process:** The eBPF verifier ensures that the program is safe to run. It validates that the program meets several conditions, for example:

- The process loading the eBPF program holds the required capabilities (privileges). Unless unprivileged eBPF is enabled, only privileged processes can load eBPF programs.
- The program does not crash or otherwise harm the system.
- The program always runs to completion (i.e. the program does not sit in a loop forever, holding up further processing).

The verifier analyzes the program to assess all possible execution paths. It steps through the instructions in order, evaluating them rather than actually executing them. Each time the verifier comes to a branch, where a decision has to be made on whether to carry on in sequence or jump to a different instruction, the verifier pushes a copy of the current state of all the registers onto a stack and explores one of the possible paths. It continues evaluating the instructions until it reaches the return at the end of the program (or reaches the limit on the number of instructions it will process, which is currently one million instructions), at which point it pops



a branch off the stack to evaluate the next possible path for the program execution. If it finds an instruction attempting to access out-of-bounds memory, using uninitialized data or that could result in an invalid operation, or if the program exceeds the maximum program size or instruction count, it fails verification. This stringent checking performed by the verifier ensures that only safe programs are run by the kernel, thereby maintaining the integrity and reliability of the kernel.

**JIT Compilation:** Once the program is verified, as stated in [10], the Just-in-Time (JIT) compilation step translates the generic bytecode of the program into the machine-specific instruction set to optimize the execution speed of the program. This makes eBPF programs run as efficiently as natively compiled kernel code or as code loaded as a kernel module. Once the programs are translated to machine code, it is required to be attached to a hook, which is discussed next.

**Loading an eBPF Program (using hooks):** Currently, eBPF programs are event-driven and are run when the kernel or an application passes a certain hook point. Pre-defined hooks include system calls, function entry/exit, kernel tracepoints, network events, and several others. If a predefined hook does not exist for a particular need, it is possible to create a kernel probe (kprobe) or user probe (uprobe) to attach eBPF programs almost anywhere in kernel or user applications. Thus, the behaviour of the Linux kernel can be modified/extended at almost any point.

How it works is that an eBPF program is attached to an event. Regardless of the process that caused the event to occur, the eBPF program is triggered. For instance, as stated in [4], if you attach a program to the syscall for opening files, it will be triggered whenever any process tries to open a file. It doesn't matter whether that process was already running when the program was loaded. This is an advantage compared to upgrading the kernel and then having to reboot the machine to use its new functionality. This immediate effectiveness of eBPF technology is a huge advantage in terms of observability and security, the program gains instant visibility over the machine.

**Required Privileges:** Unless unprivileged eBPF is enabled, all processes that intend to load eBPF programs into the Linux kernel must be running in privileged

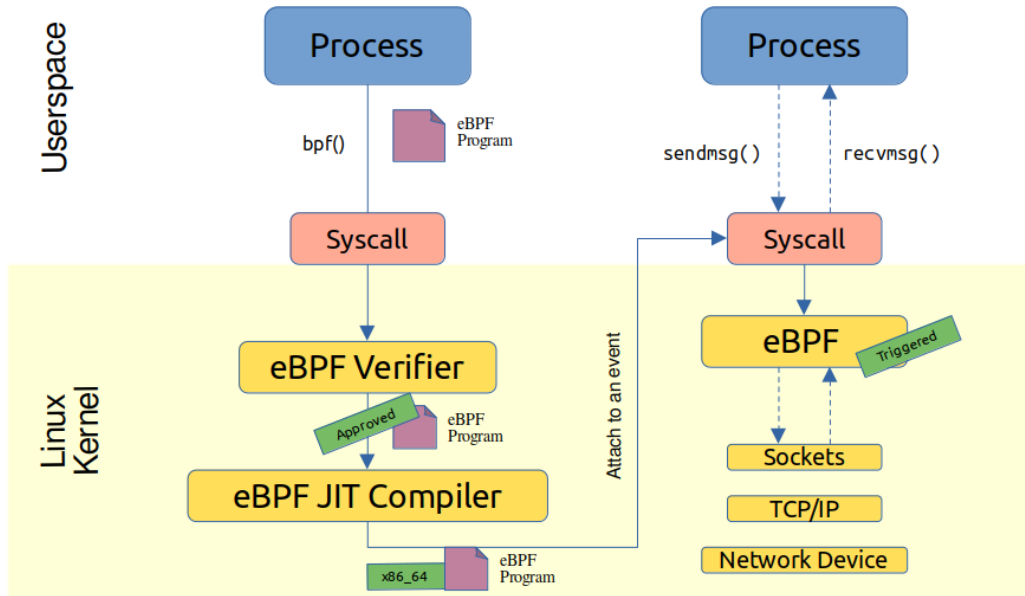


Figure 2.2: Loading and attaching to an event of an eBPF program (bytecode)

mode (root) or require the capability `CAP_BPF`. This means that untrusted programs cannot load eBPF programs.

If unprivileged eBPF is enabled, unprivileged processes can load certain eBPF programs subject to a reduced functionality set and with limited access to the kernel.

**Overview of the Workflow:** We have seen the different steps of loading an eBPF program. As illustrated in Figure 2.2, a userspace process must load the eBPF program into the kernel and this can be facilitated by the use of the `bpf()` function, which is a system call in itself. After this, the eBPF verifier verifies the program with all of its checks and throws an error if it's unsafe for the kernel. If the program is safe and passes all the checks of the verifier, the approved eBPF program (in bytecode) is then converted to native instructions by the JIT Compiler. As stated in [4], in early implementations of eBPF, the bytecode instructions were interpreted within the kernel, that is, every time an eBPF program runs, the kernel examines the instructions and converts it into machine code, which it then executes. Interpreting has since been largely replaced by JIT (just-in-time) compilation for performance reasons and to avoid the possibility of some Spectre-related vulnerabilities in the eBPF interpreter. Now, once the machine code is generated and available in the

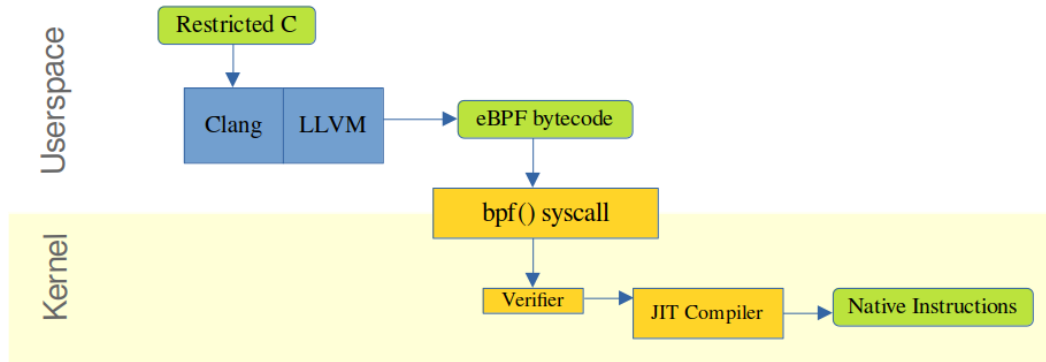


Figure 2.3: Stages of compilation of an eBPF program

kernel, this program is attached to an event, in the case of Figure 2.2, the system call `sendmsg()` and `recvmsg()`. Now, each time these system calls are used, regardless of the process using these syscalls, the eBPF program is triggered from the kernel and its execution takes place.

Now that the background of how eBPF programs are loaded and attached is understood, we shall understand the compilation process of an eBPF program by looking at its development toolchains.

## 2.2 HOW ARE EBPF PROGRAMS COMPILED?

As stated in [5], there is an ever-growing list of tools and frameworks for working with eBPF, all at varying levels of abstraction and all of them address different needs of users. However, there do seem to be a few that sit at the core of the rest of the eBPF ecosystem.

### 2.2.1 Clang

Clang is considered the standard BPF compiler. While using Clang to compile, one is required to specify the target type as `bpf`, as shown in Figure 2.4. This is made possible by Clang's use of LLVM, which has supported a BPF target since 3.7, released in 2015. This produces an object file containing BPF bytecode. From there, the program must be loaded into the kernel for the final stage. This is done via the `BPF()` syscall. Here, the program is checked for correctness and safety by

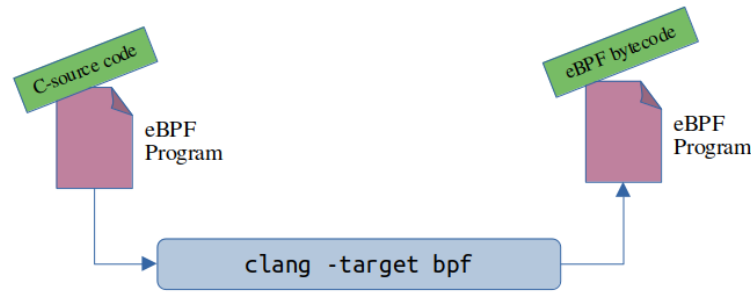


Figure 2.4: Clang compiler with target bpf specified outputs eBPF bytecode

the verifier, and then finally the JIT compiler translates BPF instructions into native machine code, all of which is shown in Figure 2.3. Although this is considered a standard for eBPF, it is unlikely that a direct compilation of eBPF programs is required. There are multiple frameworks that have abstracted this compilation process and a few of these frameworks are explained below.

### 2.2.2 BCC

BPF Compiler Collection (BCC) has been instrumental in making it easier to write BPF programs. This framework, shown in Figure 2.5 includes a suite of libraries and tools that facilitate not only the creation of BPF programs but also the userspace components necessary for loading these programs into the kernel. It enables these programs to attach to various events like tracepoints or probes and interact with shared resources such as maps. One of BCC’s most valuable features is its collection of pre-built tools, which can be used directly as command-line utilities without requiring any knowledge of writing BPF programs.

Within the BCC ecosystem, the terms “backend” and “frontend” are often used to describe different components of the BPF workflow:

- Backend: This refers to the actual BPF program, typically written in a restricted version of C. The frontend component compiles this code into BPF bytecode.
- Frontend: This userspace component is responsible for compiling and loading the BPF program, attaching it to tracepoints or probes, and managing

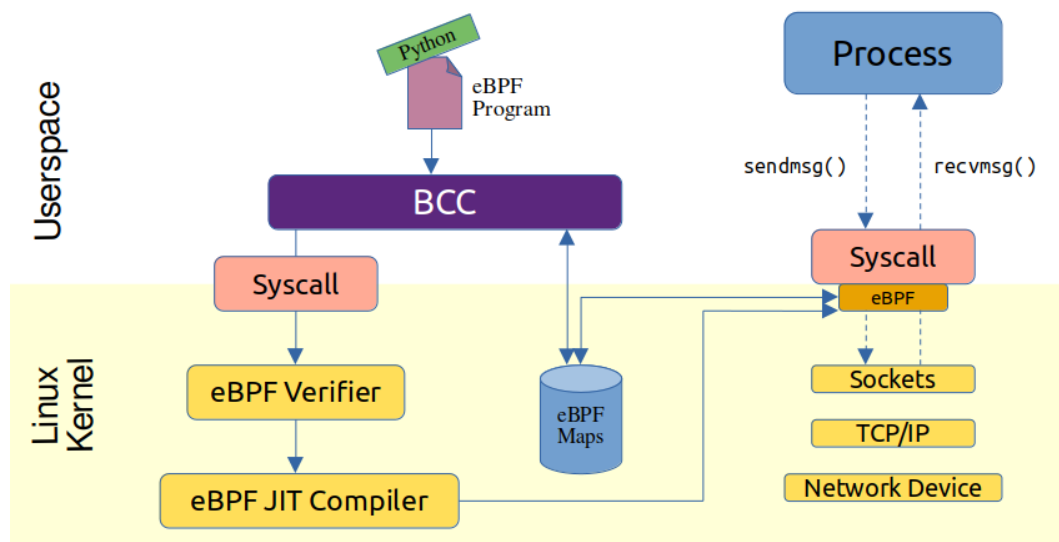


Figure 2.5: BCC compiler overview

interactions with shared resources like maps.

BCC stands out as one of the pioneering frontends for BPF, providing a robust framework that includes Python bindings. These bindings allow developers to perform frontend tasks and embed BPF programs as string literals within Python scripts, simplifying the creation of BPF-powered tools. The BCC project also includes a significant collection of pre-built tools, meaning there's a high probability that an existing tool can meet your needs, or at least serve as a close starting point. However, using BCC comes with certain drawbacks:

- **Large Footprint:** The compilation of a BPF program written with BCC occurs at runtime. This requires clang/llvm to be available on every machine where the program is run, which is why these tools are included in 'libbcc', the C library behind BCC's Python bindings. Additionally, BCC requires kernel header files to be installed on the machine, resulting in a substantial disk and memory footprint even for simple tools.
- **Delayed Compilation\*\*:** Since compilation happens at runtime, any errors, even simple ones, are only detected after deploying and running the tool, which can lead to a poor development experience.

Despite these drawbacks, BCC remains a valuable resource with many tools built on top of it, and many BPF developers have established workflows to mitigate its limitations. While BCC is not expected to disappear anytime soon, there is a growing consensus to guide eBPF newcomers towards other, more modern options. Consequently, the BCC Python bindings are now considered deprecated, and efforts are underway to convert BCC's pre-built tools to use 'libbpf' instead.

### 2.2.3 LIBBPF

Similar to BCC, libbpf is a framework designed to facilitate the development of BPF (Berkeley Packet Filter) programs. It provides a C library that not only compiles and loads BPF programs, as shown in Figure 2.6, but also handles various frontend tasks previously managed by BCC's Python bindings. However, libbpf distinguishes itself from BCC in several significant ways:

- **Kernel Integration:** libbpf is maintained as part of the upstream Linux kernel, making it the same library that kernel developers use to create BPF programs. This integration ensures compatibility across multiple kernel versions, meaning that if you develop programs using libbpf, your work is more likely to stay aligned with ongoing changes in the Linux kernel.
- **Reduced Dependencies:** Unlike BCC, libbpf does not require Clang/LLVM or kernel headers to be available on the machine running the BPF program. Compilation occurs once during development, resulting in a significantly smaller storage and memory footprint for BPF-based tools.

A key advantage of libbpf is its ability to compile BPF programs once and run them on various Linux kernel versions. This raises a critical consideration: ensuring the portability of BPF programs, particularly those that access internal kernel structures, which can vary significantly between kernel versions. libbpf addresses this challenge with two innovative technologies:

- **Compile Once - Run Everywhere (CO-RE):** CO-RE is a series of helper functions and macros within libbpf, coupled with enhancements to the Clang com-

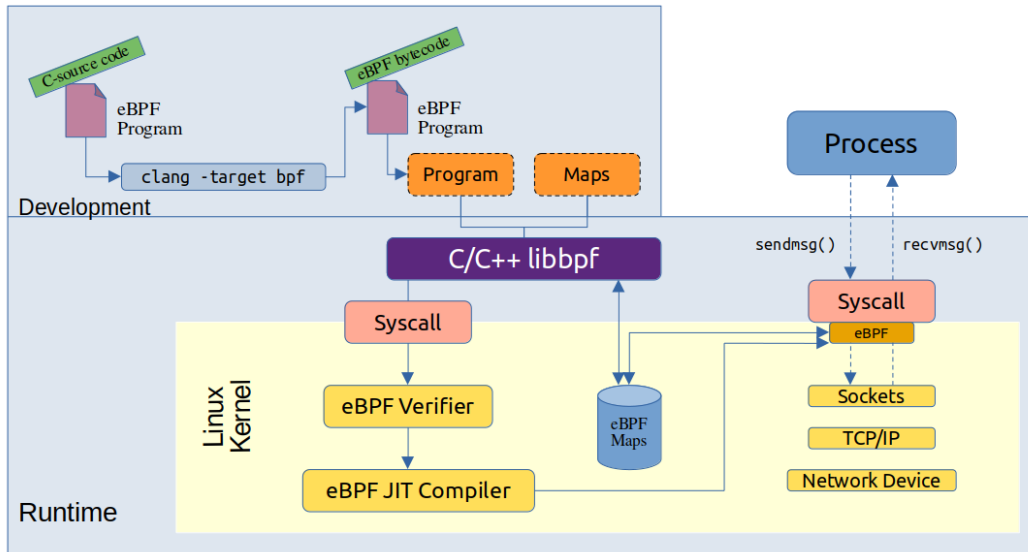


Figure 2.6: Workflow of libbpf compilation

piller. This technology allows the resulting program to emit “field relocations” alongside the compiled program, ensuring compatibility across different kernel versions.

- **BPF Type Format (BTF):** BTF provides Linux type information in a compact format, similar to the role of kernel headers but with a much-reduced footprint. This makes it feasible to include type information by default, a practice now adopted by many popular Linux distributions.

By leveraging these advanced features, libbpf simplifies the development of robust and portable BPF programs, providing a streamlined and efficient framework well-suited for modern kernel development.

#### 2.2.4 BPFTRACE

While collections like BCC offer a wide array of tools, there are often unique use cases that these tools do not address. For users who may lack the expertise or inclination to write BPF programs from scratch, bpftrace presents an advantageous middle ground. Similar to how `awk` provides a flexible yet straightforward scripting language for text processing, bpftrace offers a powerful and user-friendly interface

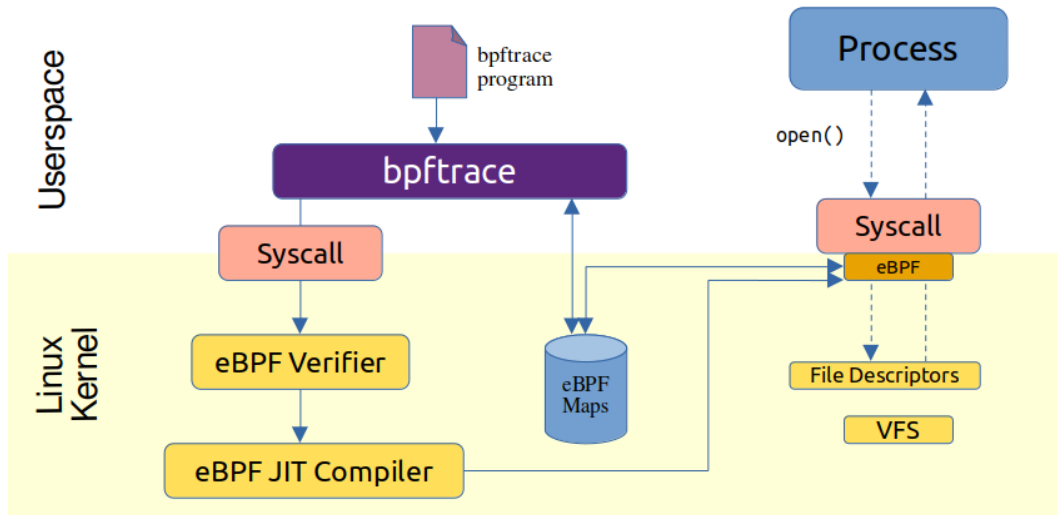


Figure 2.7: bpftool compilation overview

for creating custom BPF tracing tools, shown in Figure 2.7.

At the core of BPF programs is the execution of custom logic in response to system events. These events can vary widely, and the specific actions taken in response are determined by the use case and the developer's requirements. bpftool simplifies the process of capturing and responding to these events, allowing users to write effective tracing scripts with minimal effort.

bpftool has gained significant traction due to its ability to empower non-specialist developers to create tailored tracing tools. It eliminates the need to develop new solutions from scratch for each use case, providing a reusable framework that streamlines the development process. For many simple to moderate tracing needs, bpftool offers a robust and efficient solution.

Thus, effectively bpftool fills a crucial niche by offering a flexible yet accessible platform for BPF programming. It enables users to leverage the power of BPF without requiring deep kernel development expertise, thereby broadening the accessibility and applicability of BPF technology. This makes bpftool an invaluable tool for those looking to implement custom tracing solutions in an efficient and effective manner.



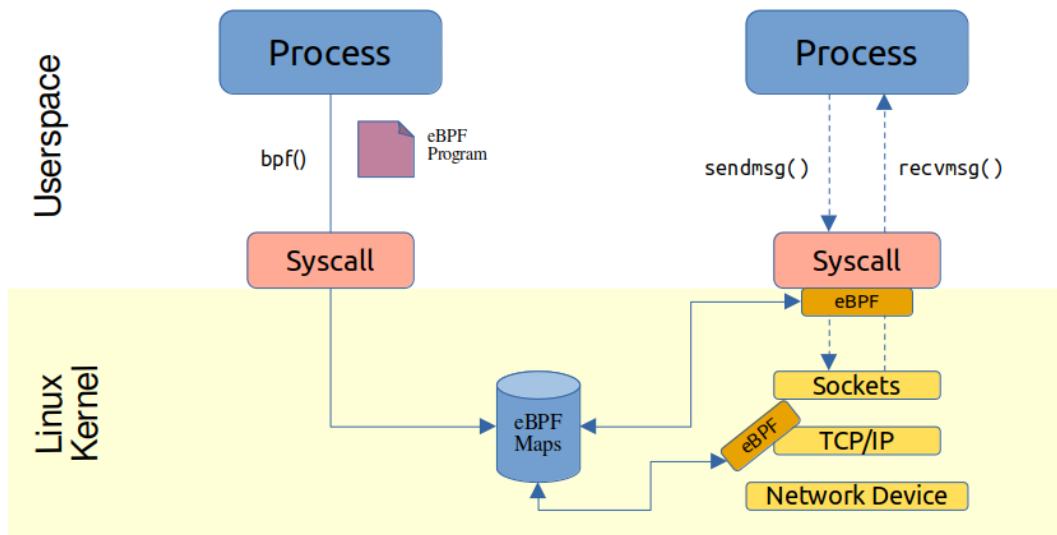


Figure 2.8: eBPF maps accessed by eBPF programs and userspace application

## 2.3 MAPS

One of the most important parts of eBPF technology is eBPF Maps. This is the way eBPF programs can communicate with each other and also how users can retrieve information from the eBPF programs in the kernel space. eBPF maps can also be used to store the state. Programs can leverage this concept to store and retrieve data in a wide set of data structures. These maps can be accessed from eBPF programs as well as from applications in userspace through system calls, as shown in Figure 2.8. As part of the restrictions posed by the verifier on eBPF programs, the maps have a limited size and hence, if the data required to be stored is higher than the maximum limit, multiple maps must be used to store the collective data.

## 2.4 HELPER FUNCTIONS

eBPF (extended Berkeley Packet Filter) helper functions are a set of predefined functions that eBPF programs can call to perform various tasks. These helper functions provide an interface to kernel functionalities, allowing eBPF programs to interact with the kernel and perform operations.

- **Safety and Security:** Helper functions ensure that eBPF programs can per-

form complex operations without compromising the security and stability of the kernel. They encapsulate functionality that requires privileged access, ensuring that eBPF programs cannot perform unsafe operations.

- Convenience: They provide a simplified interface to kernel functions, making it easier to write eBPF programs without needing extensive knowledge of kernel internals.
- Efficiency: Helper functions are optimized for performance, ensuring that eBPF programs can execute tasks quickly and efficiently.

## 2.5 TC INTERFACE

The TC interface is a network interface that handles the traffic control subsystem in Linux. As stated in [7]. TC is a powerful, yet complex framework. It relies on the notions of “queueing disciplines” (qdiscs), “classes”, “classifiers” (filters), and actions. A very simplified description might be the following:

- The user defines a qdisc, a shaper that applies a specific policy to different classes of traffic. The qdisc is attached to a network interface (ingress or egress).
- The user defines classes of traffic, and attaches them to the qdisc.
- Filters are attached to the qdisc. They are used to classify the traffic intercepted on this interface and to dispatch the packets into the different classes. A filter is run on every packet, and it can return one of the following values:
  - 0, which denotes a mismatch (for the default class configured for this filter). Next filters, if any, are run on the packet.
  - -1, which denotes the default classid configured for this filter,
  - Any other value will be considered as the class identifier referring to the class where the packet should be sent, thus allowing for non-linear classification.

- Additionally, an action to be applied to all matching packets can be added to a filter. For example, selected packets could be dropped, or mirrored on another network interface, etc.
- New nested qdiscs can be attached to the classes, and receive classes in their turn. The complete policy diagram is in fact a tree spanning under the root qdisc. But we do not need this information for the rest of the article.

Used as actions, eBPF programs behave differently, their possible return values indicating what action should actually be performed on the packet:

- `TC_ACT_UNSPEC` (-1): Use the default action configured from tc (similarly as returning -1 from a classifier).
- `TC_ACT_OK` (0): Terminate the packet processing pipeline and allow the packet to proceed.
- `TC_ACT_RECLASSIFY` (1): Terminate the packet processing pipeline and start classification from the beginning.
- `TC_ACT_SHOT` (2): Terminate the packet processing pipeline and drops the packet.
- `TC_ACT_PIPE` (3): Iterate to the next action, if available.

We use this interface to attach our eBPF program, along with direct-action, which is explained in Section 3.3.

## **Chapter 3**

### **PROPOSED CLASSIFIER IN EBPF**

Our primary goal is to build an Intrusion Detection System using ML in eBPF. For this, we have separated the entire mechanism into various steps: Data Collection, Offline Training of the Decision Tree and finally Loading the DT model into the kernel. In this chapter, we will go through the basic network and system requirements, understand the problem statement at a deeper level, and discuss the proposed idea for the solution.

#### **3.1 DATA COLLECTION**

The first step is to collect the required data from packets and store them using eBPF programs. We leverage the visibility of eBPF present at the kernel-level to read through packets and obtain essential information. The second step is to read this information stored by the eBPF program and write it onto a CSV file so that it can be used to train the DT model.

Firstly, the TC (Traffic Control) networking framework is chosen to load the eBPF program instead of XDP (eXpress Data Path). Although XDP processes network packets at the driver level, allowing for fast processing and minimal latency, this means that the packet must be dealt with before it is allocated any memory/data structure at the kernel-level, implying that a custom-made structure along with parsing functions must be used to process these packets. On the other hand, although TC is involved with more complex policies and operations, it operates at the qdisc (queuing discipline) layer and can process both ingress and egress packets. These packets, when entering the TC interface, are allocated kernel memory in the networking stack and handled by the kernel networking subsystem, and hence, allow us to use this allocated data structure and associated functions. As a result, the TC

```

struct __sk_buff {
    __u32 len;
    __u32 pkt_type;
    __u32 mark;
    __u32 queue_mapping;
    __u32 protocol;
    __u32 vlan_present;
    __u32 vlan_tci;
    __u32 vlan_proto;
    __u32 priority;
    __u32 ingress_ifindex;
    __u32 ifindex;
    __u32 tc_index;
    __u32 cb[5];
    __u32 hash;
    __u32 tc_classid;
    __u32 data;
    __u32 data_end;
    __u32 napi_id;

    /* Accessed by BPF_PROG_TYPE_sk_skb types from here to ... */
    __u32 family;
    __u32 remote_ip4; /* Stored in network byte order */
    __u32 local_ip4; /* Stored in network byte order */
    __u32 remote_ip6[4]; /* Stored in network byte order */
    __u32 local_ip6[4]; /* Stored in network byte order */
    __u32 remote_port; /* Stored in network byte order */
    __u32 local_port; /* stored in host byte order */
    /* ... here. */

    __u32 data_meta;
    __bpf_md_ptr(struct bpf_flow_keys *, flow_keys);
    __u64 timestamp;
    __u32 wire_len;
    __u32 gso_segs;
    __bpf_md_ptr(struct bpf_sock *, sk);
    __u32 gso_size;
};

```

Figure 3.1: Contents of struct `__sk_buff`

interface was chosen for this particular project. Another reason is that because TC allows more complex operations, like scheduling and traffic shaping, the extension of the current use-case of the implemented model would be easier. Transitioning the current implementation from TC to XDP does not require a great deal of change either, and thus, if latency plays a major role in the chosen use-case, the framework can be transitioned into XDP as well.

As we have understood the interface in which we will load the eBPF program, we shall move on to the program itself. Now, as the TC interface stores the packet in the networking stack, this is usable to us as a function parameter. This data structure is the struct `__sk_buff`, which stands for socket buffer. It is a fundamental data structure in the Linux kernel used to manage network packets. It represents a packet

```

struct bpf_elf_map {
    __u32 type;
    __u32 size_key;
    __u32 size_value;
    __u32 max_elem;
    __u32 flags;
    __u32 id;
    __u32 pinning;
    __u32 inner_id;
    __u32 inner_idx;
};

struct bpf_elf_map __section("maps") PUC_info = {
    .type      = BPF_MAP_TYPE_HASH,
    .id        = 1,
    .size_key   = sizeof(uint32_t),
    .size_value = sizeof(struct PUC_entry),
    .max_elem   = MAX_ENTRIES,
    .pinning    = LIBBPF_PIN_BY_NAME,
};

```

Figure 3.2: Declaration of eBPF map PUC\_info

of data that is being transmitted or received over a network interface. The `sk_buff` structure is used by the kernel to handle network packets, including their headers, data, and any necessary fragmentation or reassembly. Due to the availability of this structure, we can handle packets with more ease. A user-accessible mirror of in-kernel `sk_buff` is present in the `bpf.h` header file, as shown in Figure 3.1.

After getting the packet as an argument, using the *protocol* data member of struct `__sk_buff`, we check if the packet is an IP packet. After this check, we denote the start and end of the packet with two pointers. Then, as the ethernet header is at the beginning of `sk_buff`, we initialise a struct `ethhdr` pointer to the start of the packet. And then, we initialise a struct `iphdr` pointer to the address after incrementing the start of the pointer by the size of struct `ethhdr` to align correctly with the IP header. This is an important step as each of these structures provides us with information in their data members that we do not have to fetch by parsing through the entire packet. After the structures are placed appropriately, we fetch the packet details and store them in a map.

The definition of eBPF maps in our program is slightly different from the conventional definitions. This is necessary for eBPF programs loaded in the TC interface.

```

struct PUC_entry {
    unsigned int src_addr;
    unsigned int dest_addr;
    unsigned int src_port;
    unsigned int dest_port;
    unsigned int max_pkt_size;
    unsigned int min_pkt_size;
    unsigned int mean_pkt_size;
    unsigned long long int max_iat;
    unsigned int protocol;
    unsigned int rx_count;
    unsigned int tx_count;
    unsigned int rx_bytes;
    unsigned int tx_bytes;
    unsigned int flow_duration;
};

```

Figure 3.3: Entries of struct PUC\_entry

As you can see you Figure 3.2, the eBPF map PUC\_info (Per Unique Connection) is defined. It is of map type hash (key-value pair), where the size of key is defined as an unsigned 32-bit integer and the size of the value is the structure PUC\_entry defined in our program. The PUC\_entry structure stores all the required features for a connection, which is required for the ML model, shown in Figure 3.3 There is a separate structure defining the contents of any eBPF map, which is struct bpf\_elf\_map and then we initialize PUC\_info in the "maps" section by using the \_\_section MACROS. Note that .pinning is set as LIBBPF\_PIN\_BY\_NAME, which implies that the map, upon first initialization, will be pinned at the location /sys/fs/bpf/ by the name of the map. This allows us to access it from a different program independently.

Now, as we are collecting information of each and every unique connection, if a new connection is detected, a new entry is created in the map, and information is stored. Otherwise, the existing entry is updated. This way, we collect the required amount of data for training our Decision Tree. In our case, the dataset[11] consists of labeled network flows, including full packet payloads in PCAP format, the corresponding profiles, and the labeled flows and CSV files for machine learning purposes and we will be making use of the same.

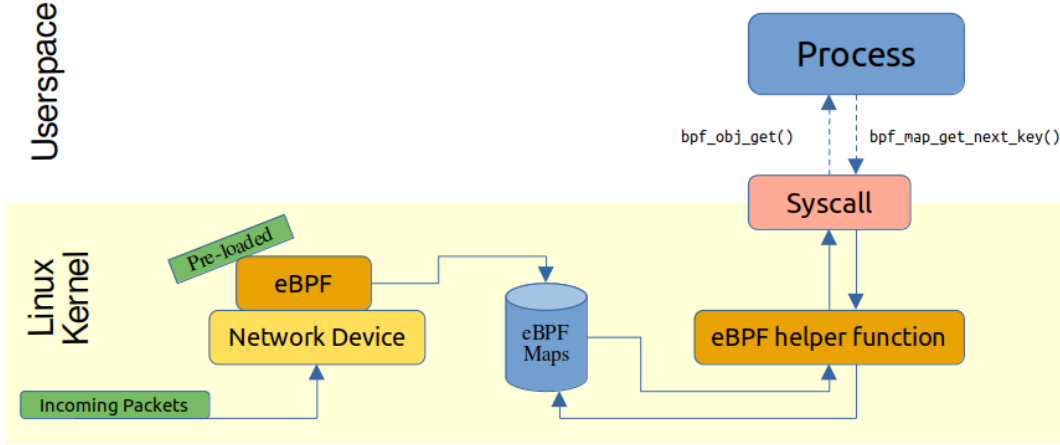


Figure 3.4: Workflow of Data Collection from eBPF and userspace program

The next step is to have a userspace program to read the data stored in the eBPF maps and write it into a CSV file. For this, we use the eBPF helper functions. As the map is pinned by name, we use the helper function `bpf_obj_get(const char *pathname)` where *pathname* is the path of the map, which is `"/sys/fs/bpf/PUC_info"`. Using this helper function, the file descriptor to the map is obtained. We then use the helper function `bpf_map_get_next_key(int fd, const void *key, void *next_key)`, where the map's file descriptor is passed along with *key* being the current key is passed (must be NULL for the first element) and the next element's key is written in *next\_key*. This way, we iterate through all the elements of the map and write it into a CSV file. Both these steps are illustrated in Figure 3.4.

### 3.2 OFFLINE TRAINING OF DT MODEL

The classifier we build in eBPF is a Decision Tree. As discussed earlier, as eBPF is not Turing-complete, ML models like Decision Trees and Neural Networks can be implemented in eBPF. We have chosen DTs due to their simplicity and effectiveness in IDSs [3]. As we have the dataset in CSV format for ML use-cases, we have used that to train the DT model. The features used for the classification are the source and destination addresses and ports, maximum, minimum, and mean packet size, maximum inter-arrival time, first and last time a packet arrived, count



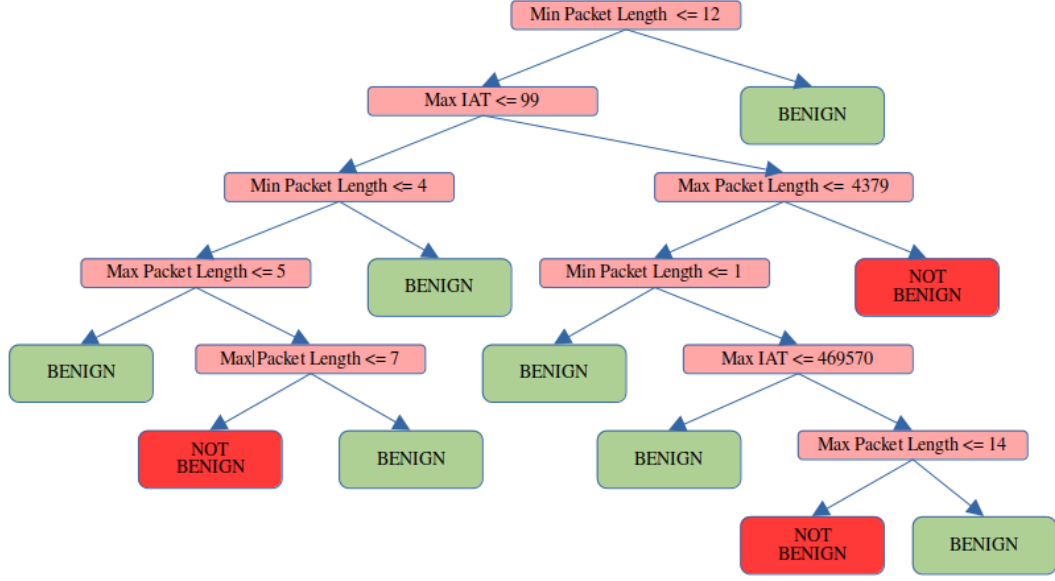


Figure 3.5: Trained Decision Tree diagram

and bytes packets, and count of TCP and UDP packets. All these features are data members of the PUC\_entry structure as well, as seen in Figure 3.3. We use all these features and train the DT model offline using scikit-learn. Now, as the eBPF programs have constraints posed by the verifier, the size of the tree must be fixed. For our specific use-case, we customize the Decision Tree to be very small in order to avoid any errors in terms of program size and to run loops. Hence, we limit the number of leaf nodes to 10 and the visualization of the DT is shown in Figure 3.5. Once the DT model is trained with the data, we look at the internal nodes and observe the features and weights of the Decision Tree. This is then stored so that in the next stage while loading the model into the kernel, the model developed for the eBPF program can be hard-coded according to the weights and features used for offline training. The ideal way would be to store the weights of the Decision Tree during offline training, store them in a file and while loading the eBPF program, these weights are stored into eBPF maps which can be read by the loaded eBPF program (model) to make decisions. This allows for updating of the weights without unloading the model from the kernel. However, for our use-case, as we are trying to keep the model minimal, we shall focus on a simple implementation.

### 3.3 LOADING DT MODEL

Now, for loading the model into the kernel, we write an eBPF program by hard-coding the weights and features used for making decisions. Hence, it is effectively a rule-based decision making with respect to the eBPF program but the rules are decided offline based on ML techniques, which in our case is the Decision Tree. This allows us to keep the model straightforward and the eBPF program to be verified. Now, as shown in Figure 3.5, the features required are minimum and maximum packet length and maximum IAT. Thus, we obtain these features alone, and using if-else conditions, we build the model. Now, when a packet is received by the program, the program runs through the if conditions to determine whether the packet is benign or not. The result is written into an eBPF map which is later read by a userspace program and written into a CSV file (similar to the Data Collection stage shown in Figure 3.4) to determine the accuracy of the model. Now, to load the program, we use the concept of direct-action. As stated in [7], the direct-action flag, used at filter attach time, tells the system that the return value from the filter should be considered as the one of action instead. This means that an eBPF program attached as a TC classifier can now return TC\_ACT\_SHOT, TC\_ACT\_OK, or another one of the reserved values. It is interpreted as such: no need to add another TC action object to drop or mirror the packet. In terms of performance, this is also more efficient, because the TC subsystem no longer needs to call into an additional action module external to the kernel. The qdisc used for loading eBPF program is the "clsact" Qdisc. which is basically a superset of the ingress qdisc, in the sense that it allows eBPF programs with direct-action flag enabled to be attached to the egress path.

Now, we compile our eBPF program with the model using the command (similar to what was discussed in Section 2.2):

```
$ clang -O2 -emit-llvm -c tc_ingress.bpf.c -o - | llc \
    -march=bpf -mcpu=probe -filetype=obj -o tc_ingress.bpf.o
```

Now, to load the object file, we need to first create a qdisc to attach the filter to. We achieve that and load the program with the following commands:

```
# tc qdisc add dev eth0 clsact
# tc filter add dev eth0 ingress bpf direct-action \
    obj foo.o sec .text
# tc filter show dev eth0 ingress
filter protocol all pref 49152 bpf chain 0
filter protocol all pref 49152 bpf chain 0 handle 0x1
tc_ingress.bpf.o:[.text] direct-action not_in_hw
id 170 tag 08cbd91cfba9c0dc jited
```

Note that the loaded program (tc\_ingress.bpf.o) appears in the second line of output indicating that the program is loaded. The direct-action flag appears in the output as well, thus, this program is enough to run both classification and action selection on the traffic.

Now, to unload the program, we use the command:

```
# tc qdisc del dev eth0 clsact
```

## 3.4 CHALLENGES FACED

As discussed, the eBPF framework is a revolutionary technology that has enabled us to achieve better and faster functionalities through the Linux kernel than was intended. Although eBPF facilitates us with implementations, various challenges were encountered along the way and a great deal of research and trial-and-error solutions were tried to fix these issues. To prevent others who are interested in the future work of this field and extensions to this project from going through the same difficulties, I intend to explain my research and findings in detail. To gain a bit more insight into the challenges faced, I shall document some of them in this report briefly.

### 3.4.1 Choice of eBPF Toolchain

A major challenge at the beginning of this project was to choose the right toolchain for our use-case. As discussed in Section 2.2, there are various ways to

compile and load an eBPF program. As stated in [6], since the inception of eBPF, it's been a constant priority for the BPF community to simplify BPF application development as much as possible, make it as straightforward and familiar of an experience as it would be for a user-space application. And with the steady progress around BPF programmability, writing BPF programs has never been easier.

Initially, starting with learning eBPF [4], the project began with the usage of the BPF Compiler Collection (BCC) which is integrated with Python. BCC provided a built-in Clang compiler that could compile your BPF code in runtime, tailoring it to a specific target kernel on the host. It was the choice to develop a maintainable BPF application that dealt with kernels that are constantly changing.

- But with BCC, you embed your BPF program C source code into your user-space program (control application) as a plain string. Although this seems like a smooth process, a huge problem in this compilation process is the debugging part. Even the most trivial compilation errors occur only at runtime and the userspace application must be recompiled and restarted each time. This does not help with fast iteration and certainly slows down the development. As eBPF was a relatively new topic to begin with and familiarity with the framework was lacking, this initial choice of toolchain was not ideal.
- Another issue was that, when the control application is eventually deployed and executed on the target host, BCC invokes its embedded Clang/LLVM, pulls in local kernel headers, and performs compilation on the fly. This assumes that the target system will have kernel headers present, which was not the case. The main issue with regard to this was that initially, the project was developed in different systems but with the same files and this had a great deal of inconsistencies in terms of compilation issues and program behaviour.

Thus, instead of spending the majority of time debugging, the toolchain used for development was switched to eunomia-bpf [9]. It is an open-source dynamic loading runtime and a compile toolchain framework. Using an executable file, the compilation and loading of eBPF programs were made easier. But customizing according to

our needs with this toolchain was difficult. Our requirement to attach the program to the ethernet interface was difficult to achieve with eunomia and its behaviour was not consistent.

At this point, a decision was made to use system calls directly without depending much on toolchains to load the program. This is when the direct-action attachment of the program to the Traffic Control subsystem was done using the commands mentioned in Section 3.3. This solved our purpose without hassle and the program was loaded successfully into the ingress of the TC ethernet interface, allowing incoming packets to be seen by the program.

### **3.4.2 Design of DT**

One of the main concerns while attempting to introduce more complexity to the programs implemented in eBPF is what the verifier would allow. In a standard implementation of a Decision Tree, we would be able to get an accuracy of 1.0 with the dataset used. But a standard implementation would not be allowed by the eBPF verifier for the following reasons:

- Unbounded loops are disallowed: We cannot have a variable number of loops in our program, thus forcing us to have a fixed number of loops. But in the first implementation of the Decision Tree, the verifier, for a reason yet to be understood, disallowed even a bounded loop implementation even though the exit was guaranteed. Thus, the implementation was changed to purely an if-else condition-based model, which is the reason for the constraint on the number of leaf nodes for the decision tree.
- Limited program and stack size: Local variables of a C program are stored in the stack after its translation to an eBPF program. As the stack space is limited to only 512 bytes, it may be insufficient to store all the local variables of a program after its translation. To make sure this size wasn't exceeded, global variables can be used as an alternate.

### 3.4.3 Debugging

To write eBPF programs without major errors and debugging issues, a more fundamental understanding is required of the verifier, assembly, and tools for the framework. With the limited time to learn about eBPF and start writing programs, it became a difficult task to debug while writing programs. For instance, figure 3.6 shows the debug message for a simple eBPF program that was written to detect ping packets and drop them. The debug message majorly involves the code of an intermediate representation of the eBPF program. If one did not know what was and wasn't allowed while writing this program or did not have much knowledge about the assembly code of eBPF, it was not an easy task to understand what was going on.

```
libbpf: prog 'ping': BPF program load failed: Permission denied
libbpf: prog 'ping': -- BEGIN PROG LOAD LOG --
0: R1=ctx(off=0,imm=0) R10=fp0
0: (15) if r1 == 0x0 goto pc+15      ; R1=ctx(off=0,imm=0)
1: (61) r2 = *(u32 *)(r1 +0)         ; R1=ctx(off=0,imm=0) R2_w=pkt(off=0,r=0,imm=0)
2: (61) r1 = *(u32 *)(r1 +4)         ; R1_w=pkt_end(off=0,imm=0)
3: (1f) r1 -= r2                    ; R1_w=scalar() R2_w=pkt(off=0,r=0,imm=0)
4: (b7) r3 = 14                     ; R3_w=14
5: (2d) if r3 > r1 goto pc+10        ; R1_w=scalar(umin=14) R3_w=14
6: (69) r3 = *(u16 *)(r2 +12)
invalid access to packet, off=12 size=2, R2(id=0,off=12,r=0)
R2 offset is outside of the packet
processed 7 insns (limit 1000000) max_states_per_insn 0 total_states 0 peak_states 0 mark_read 0
-- END PROG LOAD LOG --
```

Figure 3.6: Error message for an eBPF program

## Chapter 4

### PERFORMANCE STUDY

In this section, we shall look a little more into the dataset used, how the eBPF program performs with this dataset, inferences from the results and what are the challenges faced to implement the program.

#### 4.1 DATASET USED

As mentioned earlier, we have obtained the dataset for this use-case from [11]. The major reason for choosing this dataset is that it offers both PCAP files and CSV files of the dataset. This is convenient as we require both these formats, PCAP for the eBPF program testing and CSV files for the offline training of the Decision Tree model. The dataset is split into different days of attack and the dataset Friday was chosen for this model. This dataset consists of two attacks, DDoS and Port Scan, both of which will be classified as Not Benign by the Decision Tree. The structure of the dataset, along with the split of training and test is displayed in Table 4.1. As observed, the dataset is split into a 70-30 ratio for training and testing.

| Dataset                 | BENIGN | NOT BENIGN | Total  |
|-------------------------|--------|------------|--------|
| Friday (entire dataset) | 414275 | 288923     | 703198 |
| Training Dataset        | 290011 | 202227     | 492238 |
| Test Dataset            | 124264 | 86696      | 210960 |

Table 4.1: Split of Friday dataset

#### 4.2 DESIGN AND PERFORMANCE OF eBPF MODEL

The classification that we are performing is with respect to the entire flow connection and not an individual packet. Thus, over time, as we receive packets

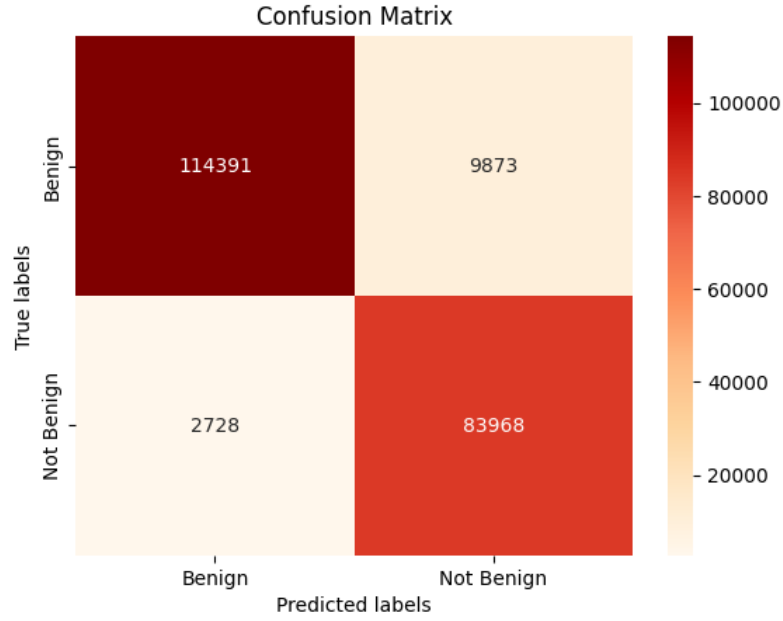


Figure 4.1: Confusion Matrix

and the maximum and minimum size of the packets with respect to a particular flow is updated along with the maximum inter-arrival time between packets, the connection is labeled as benign or not. Thus, every time a packet is received, the particular attributes of the flow are updated and then the check for whether the packet is benign or not is done. Once the flow is classified as not benign, all the packets belonging to that particular flow are dropped. Note that a flow is determined by the unique tuple consisting of source and destination address and ports along with the protocol. Although this implementation does not catch the first malicious packet to arrive due to the nature of classification, it is a step further than performing a standard classification task after training the model as there is a live updating of each flow's attributes. This design allows us to easily extend this model to catch zero-day attacks with a much better accuracy.

#### 4.2.1 Classification Report

Once all the PCAP files were sent as packets to the system with the loaded program and the final updated labels were updated, a userspace program was created to retrieve these labels and the model achieved an accuracy score of 94%. From



the confusion matrix (Figure 4.1) and classification report table (Table 4.2), we can see that with the simplistic design of only 10 leaf nodes, we are able to achieve a high precision score of 0.98 for Benign class and high recall score of 0.97 for Not Benign class.

| Class      | Precision | Recall | F1-score |
|------------|-----------|--------|----------|
| Benign     | 0.98      | 0.92   | 0.95     |
| Not Benign | 0.89      | 0.97   | 0.93     |

Table 4.2: Classification Report

This implies that if the model predicts a packet to be benign, it is accurate 98% of the time, and if the model receives malicious (not benign) packets, then 97% of the time, it classifies correctly. The required improvement for the model lies in the fact that the model classifies many benign packets as malicious, hence effectively triggering many unwanted security checks which results in an increase in the latency, or loss in the packet itself if it chooses to drop the packet. This is a common issue when machine-learning techniques are used for security to reduce false positives (in our case, false negatives as we consider benign to be positive).

#### 4.2.2 Latency

To measure latency as directly as accuracy is difficult as the classification is for the entire flow and not a single packet. Hence, latency was measured for a single packet part of a pre-existing flow. This means that given a packet is part of a particular flow, then the time is calculated from receiving the packet to determining whether the packet is benign or malicious and updating the value to be so. This is because the action after the label is updated might vary from use-case to use-case. Thus, our aim is to optimize the time taken from the entry of the packet to classifying the packet. Now, our aim is to observe a difference in the time taken for two cases:

- **In-kernel implementation of ML Model:** This is the case where the Decision Tree model is completely implemented in eBPF and loaded into the

kernel. This is in aim to reduce the time taken for the packet to reach the userspace, or in our case, the features of the packet to be read by the userspace program. With this implementation, the model obtains the features and goes through the Decision Tree in the kernel, and updates the label of the flow, whether Benign or Not Benign. After the label is updated, as shown in Figure 4.2, the total time taken is calculated and updated in the map, so that we can collect it later from a userspace program.

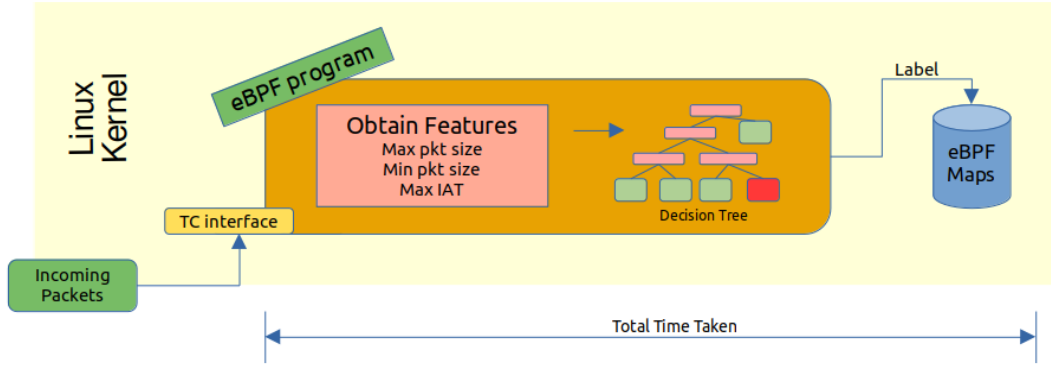


Figure 4.2: Span of total time for in-kernel implementation of model

- **Userspace implementation of ML Model:** In this case, we implement the Decision Tree in the C program in the userspace. As described in Figure 4.3, after the eBPF program receives the packet, the required features are obtained and stored in the eBPF map along with the arrival time of the packet. After this, the userspace program gets these attributes from the map, traverses the Decision Tree to classify the packet, and once the label is decided, it updates the eBPF map from userspace using helper functions. In this case, as well, the total time is calculated from the entry of the packet into the eBPF program to the point the label is updated in the map.

The average time taken is displayed in nanoseconds in Table 4.3. Note that there is nearly an 82% improvement in latency for the model implemented in eBPF compared to userspace implementation. This will allow about 5 times increase in the number of packets processed, which is a significant increase. Hence, the intended result is observed, as there is a significant improvement in the latency between both

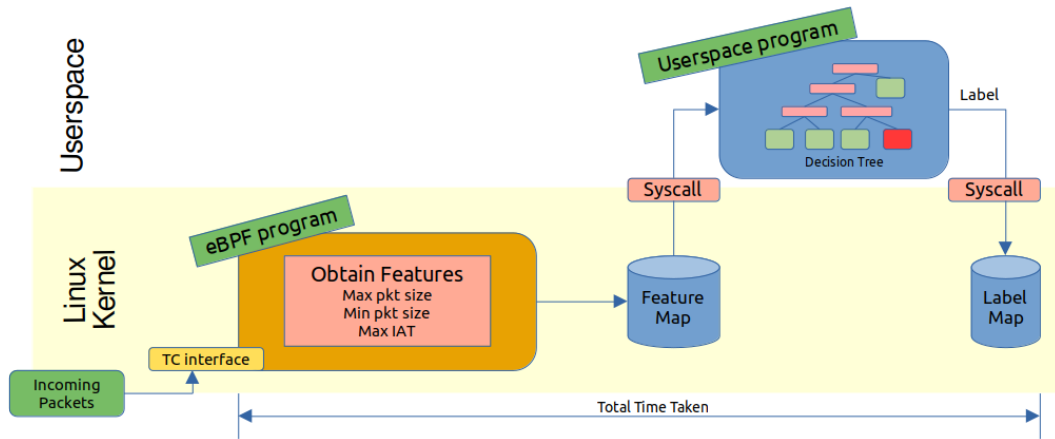


Figure 4.3: Span of total time for userspace implementation of model

| Implementation | Time Taken (in nanoseconds) |
|----------------|-----------------------------|
| Userspace      | 56648.66                    |
| Kernel         | 10557.61                    |

Table 4.3: Average Time Taken

implementations. Due to the overhead of switching context from kernel-space to userspace and the use of system calls to communicate with eBPF maps, the delay occurs. By implementing the Decision Tree in eBPF, we avoid all these overheads.

## **Chapter 5**

### **CONCLUSIONS AND FUTURE WORK**

During the course of this project, there were numerous challenges and hurdles, out of which some were solved, some were overcome and some were worked around with a compromise. A right balance between the useful functionalities that eBPF provides and all constraints posed by the framework and toolchains was found over time and the desired results were achieved. We shall look into the conclusions of this project, how it started out, and how it ended. We shall also discuss how this project is just the beginning in this line of work, how greater things can be achieved with eBPF, and how it can be extended to build better systems.

#### **5.1 CONCLUSIONS**

The problem statement was to implement an IDS using eBPF by integrating ML techniques for high-performance processing and low latency and to test the feasibility of the implementation. Although compromises had to be made with regard to the complexity of the ML model implemented in eBPF, the goal to implement and observe a performance gain (with respect to latency) and to test the feasibility of the solution in eBPF has been accomplished. The report in itself is a documentation of the feasibility of eBPF and a guidance to what tools to use for each use-case. The challenges encountered and the workarounds are essential to be listed in order to not make the same mistakes. The most important result is the observed significant improvement in the latency of the eBPF-implemented model when compared to a userspace model, which was over 80%. This result strongly suggests that this work must definitely be taken further. This implementation is only for one particular use-case. With ML introduced into eBPF, the use-cases can be vast. With that

said, we shall look into the possible future works.

## 5.2 FUTURE WORK

As the result is of significance, there is a strong reason to take this project further ahead, to implement more complex programs in eBPF. Some of the future works include:

- Due to the constraints of eBPF (and time), the simple DT model had to be implemented to obtain results. But a major improvement can be made to this as there are workarounds for each constraint. For instance, if program size is the issue, multiple eBPF programs can be written and linked together using tail-calls, a concept present in eBPF technology that allows different programs to execute in order and to switch contexts for continued execution. This is a workaround for the limit in instructions for an eBPF program.
- The accuracy of the DT model can be improved by introducing more internal and leaf nodes. The loop constraint can be worked around by bounding them and understanding what the necessary exit condition is required for the verifier to pass the eBPF program. This will help with the accuracy of the model.
- The current IDS can be extended to update the Decision Tree weights dynamically as more data is encountered by the program. This will help in improving the live classification ability of the IDS and will help realize zero-day attacks with greater accuracy.
- The implementation of ML need not stop with Decision Trees and can be extended to Random Forests and Neural Nets, which can handle more complex data and which in turn can be used for a wider set of use-cases.
- This model can be extended to solve various other use-cases that might benefit more from reduced latency and high throughput.

## REFERENCES

- [1] **Steven McCanne and Van Jacobson**, 1992, The BSD Packet Filter: A New Architecture for User-level Packet Capture. In Proceedings of the USENIX Winter 1993 Conference Proceedings on USENIX Winter 1993 Conference Proceedings (USENIX'93). USENIX Association, USA, 2.
- [2] **Alexei Starovoitov** "BPF – in-kernel virtual machine". 2015 NetDev presentation.
- [3] **F. Iglesias, D. C. Ferreira, G. Vormayr, M. Bachl, and T. Zseby**, NTARC: A Data Model for the Systematic Review of Network Traffic Analysis Research. Applied Sciences. 2020; 10(12):4307. <https://doi.org/10.3390/app10124307>
- [4] **Liz Rice**, Learning eBPF - 1st Ed., O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.
- [5] **Matt Oswalt** "The Evolving eBPF Toolchain." Matt Oswalt (blog). July 1, 2021. <https://oswalt.dev/2021/07/the-evolving-ebpf-toolchain/>
- [6] **Andrii Nakryiko** "BPF CO-RE (Compile Once – Run Everywhere)". Andrii Nakryiko's Blog, Feb 19, 2020. <https://nakryiko.com/posts/bpf-portability-and-co-re/>
- [7] **Quentin Monnet** "Understanding tc "direct action" mode for BPF", Whirl Offload (blog). April 11, 2020. <https://qmonnet.github.io/whirl-offload/2020/04/11/tc-bpf-direct-action/>
- [8] **Daniel Muller, Andrii Nakryiko**. libbpf-bootstrap. GitHub. <https://github.com/libbpf/libbpf-bootstrap>
- [9] **Ramses Rodriguez Martinez**. eunomia-bpf. GitHub. <https://github.com/eunomia-bpf/eunomia-bpf>
- [10] **eBPF Documentation**. <https://ebpf.io/what-is-ebpf/>
- [11] **Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani**, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, January 2018
- [12] **Bachl, M., Fabini, J. & Zseby, T.** A flow-based IDS using Machine Learning in eBPF. (2022)

- [13] Miano, S., Bertrone, M., Risso, F., Tumolo, M. & Bernal, M. Creating Complex Network Services with eBPF: Experience and Lessons Learned. *2018 IEEE 19th International Conference On High Performance Switching And Routing (HPSR)*. pp. 1-8 (2018)
- [14] Demoulin, H., Pedisich, I., Vasilakis, N., Liu, V., Loo, B. & Phan, L. Detecting Asymmetric Application-layer Denial-of-Service Attacks In-Flight with FineLame. *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. pp. 693-708 (2019,7), <https://www.usenix.org/conference/atc19/presentation/demoulin>
- [15] Vieira, M., Castanho, M., Pacifico, R., Santos, E., Júnior, E. & Vieira, L. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *ACM Comput. Surv.* **53** (2020,2), <https://doi.org/10.1145/3371038>